# SQL & PL/SQL Interview Preparation Guide

## Sample Schema

All SQL questions in Section 1 refer to the three related tables below.

| Table Name | Column Name | Data Type | Key / Notes |
|---|---|---|---|
| departments | dept_id | INT | Primary Key |
| departments | dept_name | VARCHAR2(50) | Name of the department |
| departments | location | VARCHAR2(50) | City where the department is based |
| employees | emp_id | INT | Primary Key |
| employees | emp_name | VARCHAR2(50) | Employee's full name |
| employees | job_title | VARCHAR2(50) | Designation, e.g. Manager, Analyst |
| employees | salary | DECIMAL(10,2) | Monthly/annual salary |
| employees | commission | DECIMAL(10,2) | Nullable — NULL if employee has no commission |
| employees | hire_date | DATE | Date of joining |
| employees | manager_id | INT | Foreign Key -> employees.emp_id (nullable, self-referencing) |
| employees | dept_id | INT | Foreign Key -> departments.dept_id (nullable) |
| employees | email | VARCHAR2(100) | Unique employee email |
| projects | project_id | INT | Primary Key |
| projects | project_name | VARCHAR2(50) | Name of the project |
| projects | dept_id | INT | Foreign Key -> departments.dept_id |
| projects | budget | DECIMAL(12,2) | Allocated project budget |
| projects | start_date | DATE | Project start date |

# Section 1: SQL Questions and Queries

Basic to advanced SQL against the schema above — filtering, aggregation, every type of join, subqueries, set operations, DDL/DML, and window functions.

## A. Basic SELECT & Filtering

| # | Question | SQL Query |
|---|---|---|
| 1 | Select all columns and all rows from employees | `SELECT * FROM employees;` |
| 2 | Select only emp_name and salary | `SELECT emp_name, salary FROM employees;` |
| 3 | Select employees earning more than 60000 | `SELECT * FROM employees WHERE salary > 60000;` |
| 4 | Select distinct job titles | `SELECT DISTINCT job_title FROM employees;` |
| 5 | List employees ordered by salary, highest first | `SELECT emp_name, salary FROM employees ORDER BY salary DESC;` |
| 6 | Get the 5 highest paid employees only | `SELECT emp_name, salary FROM employees ORDER BY salary DESC FETCH FIRST 5 ROWS ONLY; -- MySQL: ... LIMIT 5` |
| 7 | Employees whose job_title is Manager or Analyst | `SELECT * FROM employees WHERE job_title IN ('Manager','Analyst');` |
| 8 | Employees with salary between 40000 and 80000 | `SELECT * FROM employees WHERE salary BETWEEN 40000 AND 80000;` |
| 9 | Employees whose name starts with 'A' | `SELECT * FROM employees WHERE emp_name LIKE 'A%';` |
| 10 | Employees who have no manager (top of hierarchy) | `SELECT * FROM employees WHERE manager_id IS NULL;` |
| 11 | Employees who receive a commission | `SELECT * FROM employees WHERE commission IS NOT NULL;` |
| 12 | Employees earning above 50000 AND holding the title Manager | `SELECT * FROM employees WHERE salary > 50000 AND job_title = 'Manager';` |
| 13 | Employees whose job_title is NOT 'Intern' | `SELECT * FROM employees WHERE job_title <> 'Intern';` |
| 14 | Use column aliases for readability | `SELECT emp_name AS employee, salary AS monthly_pay FROM employees;` |

## B. Aggregate Functions & Grouping

| # | Question | SQL Query |
|---|---|---|
| 1 | Count total employees | `SELECT COUNT(*) FROM employees;` |
| 2 | Total salary expense across the company | `SELECT SUM(salary) FROM employees;` |
| 3 | Average salary of all employees | `SELECT AVG(salary) FROM employees;` |
| 4 | Highest and lowest salary in the company | `SELECT MAX(salary), MIN(salary) FROM employees;` |
| 5 | Count of employees in each department | `SELECT dept_id, COUNT(*) AS emp_count FROM employees GROUP BY dept_id;` |
| 6 | Departments where average salary exceeds 70000 | `SELECT dept_id, AVG(salary) FROM employees GROUP BY dept_id HAVING AVG(salary) > 70000;` |

## C. Joins (all types)

| # | Question | SQL Query |
|---|---|---|
| 1 | INNER JOIN — employee name with their department name | `SELECT e.emp_name, d.dept_name FROM employees e INNER JOIN departments d ON e.dept_id = d.dept_id;` |

| # | Question | SQL Query |
|---|---|---|
| 2 | LEFT JOIN — all employees with department name, including those with no department | `SELECT e.emp_name, d.dept_name FROM employees e LEFT JOIN departments d ON e.dept_id = d.dept_id;` |
| 3 | RIGHT JOIN — all departments with employee names, including empty departments | `SELECT e.emp_name, d.dept_name FROM employees e RIGHT JOIN departments d ON e.dept_id = d.dept_id;` |
| 4 | FULL OUTER JOIN — all employees and all departments, matched where possible | `SELECT e.emp_name, d.dept_name FROM employees e FULL OUTER JOIN departments d ON e.dept_id = d.dept_id;` |
| 5 | SELF JOIN — each employee alongside their manager's name | `SELECT e.emp_name AS employee, m.emp_name AS manager FROM employees e LEFT JOIN employees m ON e.manager_id = m.emp_id;` |
| 6 | CROSS JOIN — every employee paired with every department (Cartesian product) | `SELECT e.emp_name, d.dept_name FROM employees e CROSS JOIN departments d;` |
| 7 | Three-table JOIN — employee, their department, and that department's projects | `SELECT e.emp_name, d.dept_name, p.project_name FROM employees e JOIN departments d ON e.dept_id = d.dept_id JOIN projects p ON d.dept_id = p.dept_id;` |
| 8 | JOIN + WHERE — employees working out of the 'Bangalore' location | `SELECT e.emp_name FROM employees e JOIN departments d ON e.dept_id = d.dept_id WHERE d.location = 'Bangalore';` |
| 9 | JOIN + GROUP BY — employee count per department, by department name | `SELECT d.dept_name, COUNT(e.emp_id) AS emp_count FROM departments d LEFT JOIN employees e ON d.dept_id = e.dept_id GROUP BY d.dept_name;` |
| 10 | JOIN + aggregate — total project budget per department | `SELECT d.dept_name, SUM(p.budget) AS total_budget FROM departments d JOIN projects p ON d.dept_id = p.dept_id GROUP BY d.dept_name;` |
| 11 | Anti-join — departments that currently have no employees | `SELECT d.dept_name FROM departments d LEFT JOIN employees e ON d.dept_id = e.dept_id WHERE e.emp_id IS NULL;` |
| 12 | Self join with condition — employees earning more than their own manager | `SELECT e.emp_name FROM employees e JOIN employees m ON e.manager_id = m.emp_id WHERE e.salary > m.salary;` |

## D. Subqueries & Set Operations

| # | Question | SQL Query |
|---|---|---|
| 1 | Employees earning more than the company average (subquery in WHERE) | `SELECT emp_name, salary FROM employees WHERE salary > (SELECT AVG(salary) FROM employees);` |
| 2 | Department-wise average salary (subquery in FROM) | `SELECT dept_id, avg_sal FROM (SELECT dept_id, AVG(salary) AS avg_sal FROM employees GROUP BY dept_id) t;` |
| 3 | Employees earning more than their own department's average (correlated subquery) | `SELECT emp_name FROM employees e1 WHERE salary > (SELECT AVG(salary) FROM employees e2 WHERE e2.dept_id = e1.dept_id);` |
| 4 | Departments that have at least one project (EXISTS) | `SELECT dept_name FROM departments d WHERE EXISTS (SELECT 1 FROM projects p WHERE p.dept_id = d.dept_id);` |
| 5 | Departments that have no project (NOT EXISTS) | `SELECT dept_name FROM departments d WHERE NOT EXISTS (SELECT 1 FROM projects p WHERE p.dept_id = d.dept_id);` |
| 6 | Combine employee names and project names into one distinct list (UNION) | `SELECT emp_name AS name FROM employees UNION SELECT project_name FROM projects;` |
| 7 | Same as above but keep duplicates (UNION ALL) | `SELECT emp_name AS name FROM employees UNION ALL SELECT project_name FROM projects;` |
| 8 | dept_ids that have both employees and projects | `SELECT dept_id FROM employees INTERSECT SELECT dept_id` |

| # | Question | SQL Query |
|---|---|---|
| | (INTERSECT) | `FROM projects;` |
| 9 | dept_ids that have employees but no projects (MINUS) | `SELECT dept_id FROM employees MINUS SELECT dept_id FROM projects; -- SQL Server/Postgres: EXCEPT` |
| 10 | Categorize employees into salary bands (CASE WHEN) | `SELECT emp_name, CASE WHEN salary >= 80000 THEN 'High' WHEN salary >= 40000 THEN 'Medium' ELSE 'Low' END AS band FROM employees;` |

## E. DDL & Constraints

| # | Question | SQL Query |
|---|---|---|
| 1 | Create the departments table | `CREATE TABLE departments (dept_id INT PRIMARY KEY, dept_name VARCHAR2(50), location VARCHAR2(50));` |
| 2 | Create the employees table with foreign keys | `CREATE TABLE employees (emp_id INT PRIMARY KEY, emp_name VARCHAR2(50), job_title VARCHAR2(50), salary DECIMAL(10,2), commission DECIMAL(10,2), hire_date DATE, manager_id INT REFERENCES employees(emp_id), dept_id INT REFERENCES departments(dept_id), email VARCHAR2(100));` |
| 3 | Create the projects table with a foreign key | `CREATE TABLE projects (project_id INT PRIMARY KEY, project_name VARCHAR2(50), dept_id INT REFERENCES departments(dept_id), budget DECIMAL(12,2), start_date DATE);` |
| 4 | Add a new column to employees | `ALTER TABLE employees ADD phone VARCHAR2(15);` |
| 5 | Drop a column from employees | `ALTER TABLE employees DROP COLUMN phone;` |
| 6 | Rename a column | `ALTER TABLE employees RENAME COLUMN emp_name TO full_name;` |
| 7 | Ensure salary can never be negative (CHECK) | `ALTER TABLE employees ADD CONSTRAINT chk_salary CHECK (salary > 0);` |
| 8 | Ensure email values are unique | `ALTER TABLE employees ADD CONSTRAINT uq_email UNIQUE (email);` |
| 9 | Create an index to speed up lookups by dept_id | `CREATE INDEX idx_emp_dept ON employees(dept_id);` |
| 10 | Remove all rows from projects but keep its structure | `TRUNCATE TABLE projects;` |

## F. DML & Transaction Control

| # | Question | SQL Query |
|---|---|---|
| 1 | Insert a new department | `INSERT INTO departments (dept_id, dept_name, location) VALUES (10, 'IT', 'Bangalore');` |
| 2 | Insert a new employee | `INSERT INTO employees (emp_id, emp_name, job_title, salary, hire_date, dept_id) VALUES (201, 'Rahul', 'Analyst', 55000, DATE '2024-06-01', 10);` |
| 3 | Give every employee in dept 10 a 10% raise | `UPDATE employees SET salary = salary * 1.10 WHERE dept_id = 10;` |
| 4 | Delete an employee record | `DELETE FROM employees WHERE emp_id = 201;` |
| 5 | Permanently save changes made in a transaction | `COMMIT;` |
| 6 | Undo changes back to a named point in a transaction | `SAVEPOINT before_update; UPDATE employees SET salary = salary + 1000 WHERE emp_id = 201; ROLLBACK TO before_update;` |

## G. Window Functions & Advanced Queries

| # | Question | SQL Query |
|---|---|---|
| 1 | Rank employees by salary (RANK, DENSE_RANK, ROW_NUMBER) | `SELECT emp_name, salary, RANK() OVER (ORDER BY salary DESC) AS rnk, DENSE_RANK() OVER (ORDER BY salary DESC) AS dense_rnk, ROW_NUMBER() OVER (ORDER BY salary DESC) AS row_num FROM employees;` |
| 2 | Find the Nth (say 3rd) highest salary | `SELECT emp_name, salary FROM (SELECT emp_name, salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk FROM employees) WHERE rnk = 3;` |
| 3 | Find the second highest salary without a window function | `SELECT MAX(salary) FROM employees WHERE salary < (SELECT MAX(salary) FROM employees);` |
| 4 | Running total of salaries ordered by hire date | `SELECT emp_name, hire_date, salary, SUM(salary) OVER (ORDER BY hire_date) AS running_total FROM employees;` |
| 5 | Compare each employee's salary with the next-hired employee's (LEAD/LAG) | `SELECT emp_name, hire_date, salary, LAG(salary) OVER (ORDER BY hire_date) AS prev_hire_salary FROM employees;` |
| 6 | Find duplicate email values | `SELECT email, COUNT(*) FROM employees GROUP BY email HAVING COUNT(*) > 1;` |
| 7 | Count employees hired in each year | `SELECT TO_CHAR(hire_date,'YYYY') AS hire_year, COUNT(*) FROM employees GROUP BY TO_CHAR(hire_date,'YYYY');` |
| 8 | Department with the highest total salary payout | `SELECT dept_id, SUM(salary) AS total_pay FROM employees GROUP BY dept_id ORDER BY total_pay DESC FETCH FIRST 1 ROW ONLY;` |

# Section 2: PL/SQL Questions and Definitions

Covers essentially every core PL/SQL topic — blocks, cursors, procedures/functions, exceptions, triggers, packages, collections, dynamic SQL, transactions, and performance — so any topic-based question can be answered.

## A. PL/SQL Basics

| # | Question | Answer / Definition |
|---|---|---|
| 1 | What is PL/SQL? | PL/SQL (Procedural Language extension to SQL) is Oracle's procedural extension of SQL. It adds variables, control structures, loops, and exception handling so procedural logic can be combined with SQL data manipulation inside a single block. |
| 2 | What is the basic structure of a PL/SQL block? | DECLARE (optional — variables, cursors, exceptions), BEGIN...END (mandatory — executable statements), and EXCEPTION (optional — error handling), terminated with a slash (/) when run in tools like SQL*Plus. |
| 3 | What is an anonymous block? | A PL/SQL block with no name that is not stored in the database. It is compiled and executed only for that one run, consisting of an optional DECLARE section, a mandatory BEGIN...END, and an optional EXCEPTION section. |
| 4 | What is the difference between SQL and PL/SQL? | SQL is declarative and set-based, executed one statement at a time. PL/SQL is procedural, wraps SQL inside blocks, and adds variables, conditions, loops, and exception handling, so multiple statements can run as one unit. |
| 5 | What data types are available in PL/SQL? | Scalar (NUMBER, VARCHAR2, CHAR, DATE, BOOLEAN), composite (RECORD, TABLE, VARRAY), reference (REF CURSOR), and LOB types (CLOB, BLOB, NCLOB, BFILE). |
| 6 | What is %TYPE? | %TYPE lets a variable inherit the data type of a specific column or another variable dynamically, so if the underlying column definition changes, the variable updates automatically without code changes. |
| 7 | What is %ROWTYPE? | %ROWTYPE lets a variable take on the structure of an entire row from a table or cursor, creating a record whose fields match the table's or cursor's columns one-to-one. |
| 8 | How do you declare a constant in PL/SQL? | Using the CONSTANT keyword, e.g. tax_rate CONSTANT NUMBER := 0.18; A constant must be assigned a value at declaration and cannot be changed afterward. |
| 9 | What is the difference between CHAR and VARCHAR2? | CHAR is fixed-length and pads the remaining space with blanks up to its declared size. VARCHAR2 is variable-length and stores only the actual characters, making it more storage-efficient for varying text. |
| 10 | How are comments written in PL/SQL? | Single-line comments use --, and multi-line comments are enclosed between /* and */. |

## B. Control Structures & Loops

| # | Question | Answer / Definition |
|---|---|---|
| 1 | How is conditional logic written in PL/SQL? | Using IF-THEN, IF-THEN-ELSE, and IF-THEN-ELSIF-ELSE, each terminated with END IF. |
| 2 | What is the difference between a CASE statement and a CASE expression? | A CASE statement chooses between blocks of executable statements based on a condition and ends with END CASE. A CASE expression returns a single value based on a condition and is used inside another expression, such as a SELECT list. |
| 3 | What is a basic LOOP? | LOOP...END LOOP repeats a block of statements indefinitely until an explicit EXIT or EXIT WHEN condition is met inside the loop body. |
| 4 | What is a WHILE loop? | WHILE condition LOOP...END LOOP checks the condition before every iteration and continues looping only while that condition remains true. |

| # | Question | Answer / Definition |
|---|---|---|
| 5 | What is a numeric FOR loop? | FOR i IN 1..10 LOOP...END LOOP iterates a fixed number of times over a range, automatically incrementing (or with REVERSE, decrementing) the loop counter without it needing to be declared beforehand. |
| 6 | What is EXIT WHEN used for? | EXIT WHEN condition; is placed inside a basic LOOP to exit as soon as the condition becomes true, as an alternative to writing IF condition THEN EXIT; END IF; |
| 7 | What does the CONTINUE statement do? | CONTINUE (available from Oracle 11g onward) skips the remaining statements in the current loop iteration and moves directly to the next iteration, optionally guarded with CONTINUE WHEN. |
| 8 | What is the GOTO statement used for? | GOTO transfers control unconditionally to a labelled statement, written as <<label_name>>, within the same block. It is rarely used because it can make code harder to follow. |

## C. Cursors

| # | Question | Answer / Definition |
|---|---|---|
| 1 | What is a cursor? | A cursor is a pointer to the private memory area (context area) Oracle uses to execute a SQL statement, allowing row-by-row processing of a query's result set. |
| 2 | What is the difference between an implicit and an explicit cursor? | An implicit cursor is created and managed automatically by Oracle for any single SQL DML or SELECT INTO statement. An explicit cursor is declared and controlled by the developer with OPEN, FETCH, and CLOSE, typically for queries returning multiple rows. |
| 3 | What are cursor attributes? | %FOUND (true if the last fetch returned a row), %NOTFOUND (true if it did not), %ROWCOUNT (rows processed so far), and %ISOPEN (true if the cursor is currently open). |
| 4 | What are the steps to use an explicit cursor? | DECLARE the cursor with a SELECT statement, OPEN it to execute the query, FETCH rows one at a time into variables or a record, and CLOSE it once processing is complete to release resources. |
| 5 | What is a cursor FOR loop? | A cursor FOR loop implicitly opens the cursor, fetches each row into a record, and closes the cursor automatically at the end, removing the need to explicitly code OPEN, FETCH, and CLOSE. |
| 6 | What is a parameterized cursor? | A cursor declared with parameters, so the same cursor definition can be reused with different filter values passed in each time it is opened, instead of hardcoding the WHERE clause. |
| 7 | What is the FOR UPDATE clause? | FOR UPDATE, added to a cursor's SELECT statement, locks the selected rows so no other session can modify them until the transaction is committed or rolled back. |
| 8 | What is WHERE CURRENT OF used for? | Inside a cursor loop, WHERE CURRENT OF cursor_name lets an UPDATE or DELETE act on the exact row the cursor is currently positioned on, without repeating the row's WHERE condition. |
| 9 | What is a cursor variable (REF CURSOR)? | A REF CURSOR is a data type that lets a cursor be treated as a variable — it can be passed between subprograms and opened for different queries at runtime, unlike a static cursor definition. |
| 10 | What is the difference between a strong and a weak REF CURSOR? | A strong REF CURSOR is declared with a fixed return type (via %ROWTYPE or a record type), so Oracle can check compatibility at compile time. A weak REF CURSOR has no return type specified, making it more flexible but only checked at runtime. |
| 11 | What is SYS_REFCURSOR? | A predefined, weakly-typed REF CURSOR data type supplied by Oracle, commonly used to return query results from a procedure or function without having to declare a custom REF CURSOR type. |

## D. Procedures & Functions

| # | Question | Answer / Definition |
|---|---|---|
| 1 | What is a stored procedure? | A named PL/SQL block stored in the database that performs an action, can accept parameters, and is invoked as a standalone statement using CALL or EXEC. It does not have to return a value through its name. |
| 2 | What is a function in PL/SQL? | A named PL/SQL block stored in the database that must return exactly one value via a RETURN statement, and which can typically be used directly inside a SQL expression. |
| 3 | What is the difference between a procedure and a function? | A procedure performs an action and may return zero or more values only through OUT parameters. A function must return exactly one value through its name and, subject to purity rules, can be called inside a SQL statement. |
| 4 | What are the parameter modes in PL/SQL? | IN passes a value into the subprogram (read-only, the default). OUT passes a value back to the caller (write-only inside the subprogram). IN OUT both passes a value in and lets it be modified and returned. |
| 5 | Can a function have OUT parameters? | Yes, but a function with OUT or IN OUT parameters can only be called from PL/SQL, not from a SQL statement, because SQL expects functions to return their result solely through the RETURN value. |
| 6 | Can a function be called directly inside a SQL statement? | Yes, provided it follows Oracle's purity rules for SQL context — it must not modify database tables and generally should not read or write package state — so the optimizer can safely evaluate it row by row. |
| 7 | What is function/procedure overloading? | Defining multiple subprograms with the same name but different parameter lists (number, order, or data type) within the same package, so Oracle picks the right one based on the arguments passed at call time. |
| 8 | Can PL/SQL subprograms have default parameter values? | Yes. A parameter can be given a default value in its declaration using DEFAULT or :=, so the caller can omit that argument and the default is used automatically. |
| 9 | What is a recursive subprogram? | A procedure or function that calls itself, directly or indirectly, to solve a problem by breaking it into smaller instances of the same problem, terminating once a defined base condition is met. |
| 10 | What is the DETERMINISTIC keyword used for? | It marks a function as always returning the same output for the same input, which lets Oracle cache results or use the function in a function-based index; it is a hint, not strictly enforced by the compiler. |
| 11 | What is a PRAGMA in PL/SQL? | A compiler directive that gives the PL/SQL compiler special instructions, such as PRAGMA AUTONOMOUS_TRANSACTION, PRAGMA EXCEPTION_INIT, or PRAGMA SERIALLY_REUSABLE. It affects compilation, not runtime execution logic. |

## E. Exception Handling

| # | Question | Answer / Definition |
|---|---|---|
| 1 | What is exception handling in PL/SQL? | A mechanism to trap and respond to runtime errors using an EXCEPTION section in a block, allowing the program to react gracefully to errors instead of terminating abruptly. |
| 2 | What are predefined exceptions? | System-defined exceptions Oracle raises automatically for common errors, such as NO_DATA_FOUND, TOO_MANY_ROWS, ZERO_DIVIDE, VALUE_ERROR, INVALID_NUMBER, and DUP_VAL_ON_INDEX. |
| 3 | What are user-defined exceptions? | Custom exceptions declared by the programmer in the DECLARE section and explicitly raised with a RAISE statement when a business condition occurs that Oracle does not detect automatically. |
| 4 | What does the RAISE statement do? | RAISE signals that an exception has occurred, either re-raising the current exception (used bare inside a handler) or explicitly raising a named user- |

| # | Question | Answer / Definition |
|---|---|---|
| | | defined or predefined exception. |
| 5 | What is RAISE_APPLICATION_ERROR used for? | A built-in procedure that raises a custom error with a user-defined error number between -20000 and -20999 and a message, which is returned to the calling application as an ORA- error. |
| 6 | What are SQLCODE and SQLERRM? | SQLCODE returns the numeric error code of the exception currently being handled, and SQLERRM returns the corresponding error message text; both are typically used inside a WHEN OTHERS handler for logging. |
| 7 | What is the WHEN OTHERS clause? | A catch-all exception handler that traps any exception not matched by the earlier WHEN clauses in the EXCEPTION section; best practice is to still log or re-raise the error rather than silently swallow it. |
| 8 | What is PRAGMA EXCEPTION_INIT used for? | It associates a user-declared exception name with a specific Oracle error number, so a numbered error that has no predefined name can still be caught by a named WHEN clause. |
| 9 | How does exception propagation work? | If an exception raised in a nested block is not handled locally, it propagates outward to the enclosing block's exception handler; if no enclosing handler catches it either, the error propagates all the way to the calling environment. |
| 10 | Can exception handling blocks be nested? | Yes. A BEGIN...EXCEPTION...END block can be nested inside another block's executable or exception section, allowing localized error handling for a specific piece of logic. |

## F. Triggers

| # | Question | Answer / Definition |
|---|---|---|
| 1 | What is a trigger? | A stored PL/SQL block that executes automatically in response to a specified event — such as an INSERT, UPDATE, or DELETE — on a table or view, or a DDL/database-level event. |
| 2 | What is the difference between a row-level and a statement-level trigger? | A row-level trigger (FOR EACH ROW) fires once for every row affected by the triggering statement and can reference :OLD and :NEW. A statement-level trigger fires only once per statement, regardless of how many rows are affected. |
| 3 | What is the difference between a BEFORE and an AFTER trigger? | A BEFORE trigger fires before the triggering DML operation is applied, often used for validation or defaulting column values. An AFTER trigger fires once the operation has completed, often used for logging or cascading updates. |
| 4 | What is an INSTEAD OF trigger? | A trigger defined on a view, typically one based on a join, that replaces the DML statement issued against the view with custom logic, since such views usually cannot be updated directly. |
| 5 | What are :OLD and :NEW? | Pseudorecords available inside row-level triggers. :OLD holds the row's values before the change (unavailable for INSERT) and :NEW holds the values after the change (unavailable for DELETE). |
| 6 | What is a mutating table error and how can it be avoided? | ORA-04091, raised when a row-level trigger tries to read or modify the same table whose change caused it to fire, because that table's state is still being modified. It is commonly avoided using a compound trigger or by capturing the needed data in a collection at the statement level instead of querying the table directly inside the row-level trigger. |
| 7 | What is a compound trigger? | A trigger (introduced in Oracle 11g) that combines BEFORE STATEMENT, BEFORE EACH ROW, AFTER EACH ROW, and AFTER STATEMENT timing points in a single trigger body, sharing common variables — commonly used to avoid mutating table errors. |
| 8 | What are system or DDL triggers? | Triggers that fire on database-level or schema-level events rather than DML, such as CREATE, DROP, ALTER (DDL triggers), or LOGON/LOGOFF (database-level triggers), created with ON SCHEMA or ON DATABASE. |

| # | Question | Answer / Definition |
|---|---|---|
| 9 | How do you enable or disable a trigger? | ALTER TRIGGER trigger_name ENABLE; or ALTER TRIGGER trigger_name DISABLE; A disabled trigger remains defined in the database but does not fire until re-enabled. |

## G. Packages

| # | Question | Answer / Definition |
|---|---|---|
| 1 | What is a package in PL/SQL? | A schema object that groups logically related PL/SQL types, variables, cursors, procedures, and functions into a single named unit, made up of a specification and a body. |
| 2 | What is the difference between a package specification and a package body? | The specification declares the public interface — the procedures, functions, types, and variables visible to callers. The body implements those subprograms and may also contain private elements not exposed outside the package. |
| 3 | What is the difference between public and private package elements? | Public elements are declared in the package specification and can be accessed from outside the package. Private elements exist only in the package body and are visible only to code within that same package. |
| 4 | What is meant by package state / package-level variables? | Variables declared in a package specification or body (outside any subprogram) retain their values for the duration of a session, letting different calls to the package's subprograms within that session share data. |
| 5 | What is a package initialization block? | An optional BEGIN...END block at the end of the package body that runs exactly once, the first time the package is referenced in a session, typically used to set up initial values for package variables. |
| 6 | What are the advantages of using packages? | Modularity, encapsulation of implementation details, improved performance since the whole package is loaded into memory and cached together, better security through controlled visibility, and the ability to maintain session state via package variables. |
| 7 | Can subprograms inside a package be overloaded, and what is forward declaration? | Yes, procedures and functions within a package can be overloaded with different parameter signatures. Forward declaration means declaring a subprogram's signature earlier in the package body so it can be called by another subprogram defined before its own full body appears, resolving mutual-call ordering issues. |

## H. Collections & Records

| # | Question | Answer / Definition |
|---|---|---|
| 1 | What is a PL/SQL record? | A composite data type that groups related fields, possibly of different data types, under a single variable — similar to a structure — either user-defined with a TYPE...IS RECORD or derived using %ROWTYPE. |
| 2 | What are collections in PL/SQL, and what types exist? | Collections are composite data types holding multiple values of the same element type: associative arrays, nested tables, and VARRAYs. |
| 3 | What is an associative array? | An in-memory collection (also called an INDEX BY table) indexed by an integer or string, with no fixed size, that exists only for the duration of the session and is never stored directly in a database column. |
| 4 | What is a nested table? | A collection type that can grow to any size, can become sparse after elements are deleted, and — unlike an associative array — can be stored as a database column type in addition to being used in PL/SQL. |
| 5 | What is a VARRAY? | A collection with a fixed maximum size declared upfront, storing elements in a dense, ordered sequence; it can also be used as a database column type, typically for small, bounded lists. |
| 6 | What are common collection methods? | COUNT (number of elements), FIRST/LAST (lowest/highest index), |

| # | Question | Answer / Definition |
|---|---|---|
| | | NEXT/PRIOR (index navigation), EXISTS (checks if an index is defined), EXTEND (adds elements), TRIM (removes elements from the end), and DELETE (removes elements). |
| 7 | What is BULK COLLECT? | A clause that fetches all (or a batch of) rows from a SQL query directly into a PL/SQL collection in a single context switch between the SQL and PL/SQL engines, instead of fetching row by row. |
| 8 | What is FORALL? | A statement that executes a single DML statement repeatedly for every element of a collection in one context switch, dramatically improving performance over an equivalent row-by-row loop of DML statements. |
| 9 | What does the LIMIT clause do in BULK COLLECT? | It caps the number of rows fetched into the collection per BULK COLLECT call, letting large result sets be processed in manageable batches instead of loading everything into memory at once. |
| 10 | What does SAVE EXCEPTIONS do with FORALL? | It allows a FORALL statement to continue processing all elements even if some iterations raise errors, collecting the failures into the SQL %BULK_EXCEPTIONS collection for inspection after the statement completes. |

## I. Dynamic SQL & Transaction Control

| # | Question | Answer / Definition |
|---|---|---|
| 1 | What is dynamic SQL? | SQL or PL/SQL text that is constructed and executed at runtime rather than being fixed and compiled into the code, useful when table names, column names, or WHERE conditions are not known until execution. |
| 2 | What is EXECUTE IMMEDIATE? | The native dynamic SQL statement used to parse and immediately execute a dynamically built SQL or PL/SQL string, optionally passing bind values in with USING and returning results with INTO. |
| 3 | What is the difference between native dynamic SQL and DBMS_SQL? | Native dynamic SQL (EXECUTE IMMEDIATE / OPEN FOR) is simpler and used when the number and types of columns or bind variables are known in advance. DBMS_SQL is a more verbose package used when the SQL structure — such as the number of columns — is only known at runtime. |
| 4 | What is the USING clause in dynamic SQL for? | It passes bind variable values into an EXECUTE IMMEDIATE statement, and with USING OUT, retrieves output values back, keeping the dynamic SQL parameterized instead of concatenating literal values into the string. |
| 5 | What are COMMIT, ROLLBACK, and SAVEPOINT? | COMMIT permanently saves all changes made in the current transaction. ROLLBACK undoes the current transaction's changes, either entirely or back to a named SAVEPOINT, which marks an intermediate point a transaction can be rolled back to. |
| 6 | What is an autonomous transaction? | A transaction started within another transaction using PRAGMA AUTONOMOUS_TRANSACTION, which can be committed or rolled back independently of, and without affecting, the calling (parent) transaction. |

## J. Performance & Miscellaneous

| # | Question | Answer / Definition |
|---|---|---|
| 1 | What is the NOCOPY hint? | A hint applied to OUT or IN OUT parameters suggesting Oracle pass large values (like collections) by reference instead of by value, reducing copying overhead; Oracle may ignore it in some cases, so it is a hint rather than a guarantee. |
| 2 | What are bind variables and why are they important? | Placeholders (like :emp_id) used in SQL/PL/SQL instead of literal values. They let Oracle reuse the same parsed execution plan across calls with different values, reducing parsing overhead and improving performance. |

| # | Question | Answer / Definition |
|---|---|---|
| 3 | What is the WITH clause (subquery factoring)? | A clause that lets a named subquery be defined once at the start of a SQL statement and then referenced multiple times in the main query, improving readability and, in some cases, letting Oracle materialize the result once. |
| 4 | What is a pipelined function? | A table function that returns rows incrementally to the caller using PIPE ROW statements as they are produced, instead of building the entire result collection in memory before returning it. |
| 5 | What is a table function? | A function that returns a collection type, which can then be queried like a table using the TABLE() operator in a SQL FROM clause, e.g. SELECT * FROM TABLE(my_function()); |
| 6 | What is DBMS_OUTPUT.PUT_LINE used for? | A built-in procedure used to print text to a buffer that tools like SQL*Plus or SQL Developer display on screen, primarily used for debugging and tracing PL/SQL execution. |
| 7 | What is UTL_FILE used for? | A built-in package that lets PL/SQL read from and write to operating system text files through directories defined on the database server. |
| 8 | What is PL/SQL code wrapping? | A process (using the WRAP utility or DBMS_DDL.WRAP) that obfuscates PL/SQL source code into an unreadable format to protect intellectual property, while leaving it fully executable by the database. |
| 9 | What is conditional compilation? | A feature using $IF, $THEN, $ELSE preprocessor-style directives and compile-time flags to include or exclude sections of PL/SQL code at compile time, useful for maintaining version- or environment-specific logic in one source file. |
| 10 | What is the difference between native and interpreted PL/SQL compilation? | Interpreted PL/SQL compiles to a portable byte code executed by the PL/SQL engine at runtime. Native compilation compiles PL/SQL to native machine code ahead of time, which can execute faster for computation-heavy code, controlled by the PLSQL_CODE_TYPE parameter. |
| 11 | What is a global temporary table (GTT)? | A table whose definition is permanent and shared, but whose data is private to each session or transaction. ON COMMIT DELETE ROWS clears its data at commit; ON COMMIT PRESERVE ROWS keeps the data for the rest of the session. |